

Rational Matrix Policy Optimization for Rational Feed-Forward Language Models

Anonymous authors
Paper under double-blind review

Abstract

1 Introduction

2 Related Work

3 Method

Let $\mathcal{L}(\theta)$ be the language-model training objective. We study optimizers for Transformer feed-forward layers whose hidden nonlinearity is a grouped rational map. The method, Rational Matrix Policy Optimization, keeps a standard base optimizer for non-rational parameters and replaces only the update rule for the matrices surrounding each rational block. Its design uses two facts: the input and output matrices of a rational block have different functional roles, and the block has an exact positive gauge symmetry.

3.1 Rational feed-forward parameterization

For layer l , let $x_l \in \mathbb{R}^d$ be the residual-stream input. The RLB block projects x_l into G groups of width m ,

$$z_l = A_l x_l, \quad z_{l,g} = \text{group}_g(z_l), \quad (1)$$

normalizes each group by its RMS radius,

$$r_{l,g} = \sqrt{m^{-1} \|z_{l,g}\|_2^2 + \epsilon}, \quad u_{l,g} = z_{l,g} / r_{l,g}, \quad (2)$$

and applies a learned rational map in normalized coordinates,

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \quad y_l = B_l \text{concat}_{g=1}^G h_{l,g}. \quad (3)$$

The parameters around the rational map therefore have distinct meanings: A_l selects the domains seen by rational groups, $R_{l,g}$ controls nonlinear shape, and B_l recombines rational features into the residual stream.

3.2 Gauge symmetry

The group normalization makes the block invariant to a positive reparameterization. For $a_g > 0$, let $D_l(a)$ be block diagonal with g th block $a_g I_m$. Then

$$A'_l = D_l(a) A_l, \quad B'_l = B_l D_l(a)^{-1} \quad (4)$$

represents the same function as (A_l, B_l) . Indeed, $z_{l,g}$ and $r_{l,g}$ both scale by a_g , while $u_{l,g}$ is unchanged and the inverse scale in B'_l cancels the change in $h_{l,g}$. Consequently, parameter motion and function motion are not equivalent: an optimizer can change matrix norms and adaptive states while leaving the represented function fixed.

3.3 MatrixPolicy update

We partition parameters as

$$\theta = \theta_{\text{base}} \cup \theta_{\text{coeff}} \cup \{A_l, B_l\}_{l=1}^L. \quad (5)$$

The base parameters are updated by a standard optimizer, and rational coefficients are updated by a coefficient rule. MatrixPolicy is applied only to A_l and B_l . At step t , each matrix role $r \in \{\text{in}, \text{out}\}$ receives a role-depth factor $\rho_r(l)$ and an on-policy statistic factor $s_{\text{stat}}(l, t)$. The matrix update is

$$M_{l,r}^{t+1} = M_{l,r}^t + \Delta_{\text{adapt}}(M_{l,r}; \eta_t a_{\text{mat}}(l, r, t)[1 - \mu(l, r, t)]) + \Delta_{\text{mat}}(M_{l,r}; \eta_t a_{\text{matnorm}}\mu(l, r, t)), \quad (6)$$

where $M_{l,\text{in}} = A_l$, $M_{l,\text{out}} = B_l$, η_t is the shared base learning rate, Δ_{adapt} is an Adam-style adaptive step (Kingma & Ba, 2015), and Δ_{mat} is a short early matrix-normalized step. The mixture coefficient $\mu(l, r, t)$ is nonzero only in the early phase of training. After the matrix update, the optimizer applies the exact gauge transform

$$A_{l,g} \leftarrow e^{\ell_g} A_{l,g}, \quad B_{l,g} \leftarrow B_{l,g}/e^{\ell_g}, \quad (7)$$

which changes the parameterization but not the represented function. The detailed forms of ρ_r , μ , group-stat scaling, and ℓ_g are given in Appendix A.

3.4 Algorithm

Algorithm 1: Rational Matrix Policy step. Given gradients $\nabla \mathcal{L}(\theta^t)$ and base learning rate η_t :

1. Update exponential moving averages of rational-group pressures.
2. Compute role-depth and group-stat multipliers for each RLB matrix.
3. Update non-rational parameters with the base optimizer and update rational coefficients with the coefficient rule.
4. Update each A_l and B_l with the MatrixPolicy mixture step.
5. Apply the exact positive-gauge rebalance to each rational group.

4 Experiments

5 Discussion

6 Limitations

7 Conclusion

References

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In International Conference on Learning Representations, 2015. URL <https://mlnethology.org/iclr/2015/kingma2015iclr-adam/>.

A MatrixPolicy Details

Role-depth factors. For normalized layer depth $d_l = l/(L-1)$,

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(d_l - 0.5), 0.55, 1.40), \quad (8)$$

$$\rho_{\text{out}}(l) = \text{clip}(1 + 1.00(d_l - 0.5), 0.55, 1.40), \quad (9)$$

$$a_{\text{role}}(l, r) = \max(0.10, 1 + 1.20(\rho_r(l) - 1)). \quad (10)$$

The local adaptive scale is

$$a_{\text{mat}}(l, r, t) = \text{clip}(3.0 a_{\text{role}}(l, r) a_{\text{stat}}(l, r, t), 0.40, 4.0). \quad (11)$$

On-policy group statistics. For group g , MatrixPolicy tracks relative gradient pressures

$$p_{\text{in},g} = \frac{\text{rms}(\nabla A_{l,g})}{\text{rms}(A_{l,g})}, \quad p_{\text{out},g} = \frac{\text{rms}(\nabla B_{l,g})}{\text{rms}(B_{l,g})}, \quad p_{\text{rat},g} = \text{rms}(\nabla \theta_{\text{coeff},g}). \quad (12)$$

With EMAs $q_{\text{in},g}$, $q_{\text{out},g}$, and $q_{\text{rat},g}$, we define

$$\pi_g = \log q_{\text{in},g} - \log q_{\text{out},g}, \quad \alpha_g = \log q_{\text{rat},g} - \frac{1}{2}(\log q_{\text{in},g} + \log q_{\text{out},g}). \quad (13)$$

Group-stat preconditioning. The group preconditioner is centered and clipped:

$$c_g = c_{\text{gain},g} c_{\text{pressure},g} c_{\text{activity},g}, \quad c_g \leftarrow c_g / \text{geomean}(c), \quad c_g \leftarrow \text{clip}(c_g, 0.75, 1.35). \quad (14)$$

The factors are

$$c_{\text{gain},g} = \left(\frac{\text{geomean}(k)}{k_g} \right)^{0.20}, \quad c_{\text{pressure},g} = \exp(0.10 d_{\text{pressure},g}), \quad (15)$$

where $d_{\text{pressure},g} = -\pi_g$ for A_l and $+\pi_g$ for B_l . The activity damping term is

$$c_{\text{activity},g} = \exp[-0.20 \text{ReLU}((\alpha_g - 0.05)/0.45)]. \quad (16)$$

Gauge rebalance. Let

$$\gamma_g = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (17)$$

The target log-ratio is

$$\tau_g = \frac{1}{2}(\log d_g^{\text{gain}} - \log o_g^{\text{gain}}) + \beta_{\text{pressure}}(\log q_{\text{in},g} - \log q_{\text{out},g}). \quad (18)$$

The applied correction is

$$\ell_g = \frac{1}{2}(\tau_g - \gamma_g), \quad \ell_g \leftarrow \text{clip}(s(t, l) a_g^{\text{activity}} \ell_g, -\ell_{\text{max}}, \ell_{\text{max}}). \quad (19)$$

The transform $A_{l,g} \leftarrow e^{\ell_g} A_{l,g}$ and $B_{l,g} \leftarrow B_{l,g} / e^{\ell_g}$ preserves the RLB function exactly up to floating-point error.